REQUIREMENTS_LLM_VERIFIER_INTEGRATION

COMPREHENSIVE REQUIREMENTS DOCUMENT

LLMsVerifier as Single Source of Truth for HelixTranslate Model Provisioning

---

DOCUMENT METADATA

- **Project**: HelixTranslate / LLMsVerifier Integration
- **Version**: 1.0
- **Date**: 2025-05-01
- **Classification**: Exhaustive Requirements Analysis
- **Source Repositories**:
  - `git@github.com:HelixDevelopment/HelixTranslate.git` (Main project - Go 1.25.2, ~936 files, event-driven, 9 LLM providers)
  - `https://github.com/vasic-digital/LLMsVerifier` (Go module, 1,207 files, 25+ providers, verification + scoring)
  - `https://github.com/HelixDevelopment/HelixAgent` (Reference - already integrated, 51+ providers, 35 MCPs)

TABLE OF CONTENTS

---

1. FUNCTIONAL REQUIREMENTS

1.1 Single Source of Truth (SSOT) - CRITICAL PRIORITY

ID	Requirement	Priority
F-SSOT-001	LLMsVerifier SHALL be the EXCLUSIVE and ONLY source of truth for all LLM models available to HelixTranslate. No model may be used in HelixTranslate that does not originate from LLMsVerifier.	Critical
F-SSOT-002	HelixTranslate MUST NOT maintain any independent model registry, hardcoded model lists, or fallback model definitions outside of LLMsVerifier-provided data.	Critical
F-SSOT-003	All model metadata (name, ID, capabilities, token limits, pricing, features) SHALL be retrieved from LLMsVerifier at runtime or build time.	Critical
F-SSOT-004	Model information SHALL be cached locally in HelixTranslate with configurable TTL, but cache invalidation MUST trigger re-fetch from LLMsVerifier.	High
F-SSOT-005	If LLMsVerifier is unreachable, HelixTranslate SHALL either: (a) use expired cache with warning, or (b) enter degraded mode with explicit user notification. No unauthorized models SHALL be used.	Critical
F-SSOT-006	The existing `pkg/models/registry.go` in HelixTranslate SHALL be refactored to delegate all model lookups to LLMsVerifier client instead of local/hardcoded definitions.	Critical
F-SSOT-007	The existing `pkg/models/downloader.go` in HelixTranslate SHALL only download models that are verified and approved by LLMsVerifier.	Critical

1.2 Model Validation, Verification, and Scoring Gate - CRITICAL PRIORITY

ID	Requirement	Priority
F-GATE-001	ONLY models that have PASSED LLMsVerifier validation, verification, and scoring pipeline SHALL be presented to end users of HelixTranslate.	Critical
F-GATE-002	Models MUST satisfy ALL three criteria: (a) validation passed, (b) verification passed, (c) positive scoring (> 0 overall score).	Critical
F-GATE-003	The scoring algorithm defined in LLMsVerifier SHALL be applied: `Overall Score = (Responsiveness x 0.30) + (Code Capability x 0.25) + (Feature Richness x 0.25) + (Reliability x 0.20)`. Models with score <= 0 SHALL be rejected.	Critical
F-GATE-004	Models MUST have `VerificationStatus == "verified"` AND `CanSeeCode == true` to be eligible for HelixTranslate use.	Critical

ID	Requirement	Priority
F-GATE-005	Models MUST have `AffirmativeResponse == true` (positive confirmation of capability) to be eligible.	Critical
F-GATE-006	Models SHALL be filtered using the `EnhancedModelProviderService.filterVerifiedModels()` logic from LLMsVerifier's `model_provider_service_with_verification.go`.	Critical
F-GATE-007	The `VerifiedConfigGenerator.GenerateVerifiedConfig()` workflow SHALL be the canonical path for generating HelixTranslate's available model list.	Critical
F-GATE-008	End users SHALL NEVER see models that have not passed verification. UI/CLI SHALL only display verified models with their verification scores.	Critical
F-GATE-009	Each displayed model SHALL show: verification score, last verified timestamp, supported features (streaming, tool calling, vision, embeddings, MCP, LSP, ACP), and cost metrics.	High

1.3 Provider Integration - CRITICAL PRIORITY

ID	Requirement	Priority
F-PROV-001	ALL 25+ providers from LLMsVerifier SHALL be available in HelixTranslate: Anthropic, Cerebras, Cloudflare, Cohere, DeepSeek, Groq, Hyperbolic, Kilo, Kimi, KimiCode, Mistral, Modal, NLP Cloud, Novita, OpenAI, PublicAI, Qwen, Replicate, SambaNova, Sarvam, SiliconFlow, TogetherAI, Upstage, Vulavula, xAI, Zhipu.	Critical
F-PROV-002	HelixTranslate's current 9 LLM providers SHALL be expanded to incorporate ALL LLMsVerifier-supported providers.	Critical
F-PROV-003	Each provider SHALL have complete configuration: API endpoint, authentication, rate limits, model mappings, feature flags.	High
F-PROV-004	Provider authentication (API keys) SHALL be managed through HelixTranslate configuration (config file or .env file), consistent with LLMsVerifier and HelixAgent patterns.	Critical
F-PROV-005	Provider-specific model discovery SHALL use LLMsVerifier's `ModelProviderService.GetModels()` and `GetAllModels()` methods.	High
F-PROV-006	Provider health checks SHALL be performed via LLMsVerifier's verification pipeline before any model from that provider is offered.	High
F-PROV-007	Provider failover and fallback chains SHALL be validated with actual failures (per Constitution CONST-035).	High

1.4 MCP, LSP, ACP, Embeddings, RAG, Skills, and Plugins Integration - CRITICAL PRIORITY

ID	Requirement	Priority
F-EXT-001	ALL available MCPs (Model Context Protocols) SHALL be incorporated into HelixTranslate. Reference: HelixAgent has 35 MCPs across submodules.	Critical
F-EXT-002	ALL available LSPs (Language Server Protocols) SHALL be incorporated for enhanced translation quality and code-aware translation.	Critical
F-EXT-003	ALL available ACPs (Agent Communication Protocols) SHALL be incorporated per LLMsVerifier's `ACP_API_DOCUMENTATION.md`, `ACP_COMPLETION_CHECKLIST.md`, `ACP_EXAMPLES_AND_DEMOS.md`.	Critical
F-EXT-004	ALL available Embeddings models SHALL be incorporated for semantic translation, context preservation, and vector search capabilities.	Critical
F-EXT-005	ALL available RAGs (Retrieval-Augmented Generation systems) SHALL be incorporated for context-aware translation with knowledge retrieval.	Critical
F-EXT-006	ALL available Skills SHALL be incorporated for specialized translation domains (legal, medical, technical, literary).	Critical
F-EXT-007	ALL available Plugins SHALL be incorporated for extensible translation workflows.	Critical
F-EXT-008	Feature detection SHALL be performed via LLMsVerifier's verification pipeline (streaming, tool_calling, embeddings, vision, mcp, lsp, acp support testing).	Critical
F-EXT-009	Integration SHALL be FLAWLESS - each MCP/LSP/ACP/Embedding/RAG/Skill/Plugin MUST be verified working with real API calls before being offered.	Critical

1.5 API and Client Integration - HIGH PRIORITY

ID	Requirement	Priority
F-API-001	HelixTranslate SHALL integrate LLMsVerifier Go module (`digital.vasic.llmsverifier`) as a dependency in `go.mod`.	Critical
F-API-002	HelixTranslate SHALL use LLMsVerifier's `client.Client` from `llm-verifier/client/client.go` for all LLMVerifier communications.	Critical
F-API-003	LLMsVerifier API endpoints SHALL be used: `/api/health`, `/api/models`, `/api/models/{id}`, `/api/models/{id}/verify`, `/api/providers`.	High
F-API-004		High

ID	Requirement	Priority
	Authentication tokens SHALL be managed via `client.Client.SetToken()` with credentials from HelixTranslate config/.env.	
F-API-005	API key provisioning for both LLMsVerifier AND HelixAgent SHALL be through HelixTranslate configuration (config file or .env file).	Critical
F-API-006	The `VerifiedConfigGenerator` SHALL produce configuration consumed by HelixTranslate's `internal/config/config.go`.	Critical
F-API-007	HTTP client timeout SHALL be configurable (default 30s per LLMsVerifier client pattern).	Medium
F-API-008	Rate limiting SHALL be implemented using LLMsVerifier's `client.RateLimiter` pattern.	High
F-API-009	Circuit breakers SHALL be implemented for all LLMsVerifier API calls per Constitution requirements.	High
F-API-010	Retry logic with exponential backoff SHALL be implemented for transient failures.	High

1.6 Translation Workflow Integration - HIGH PRIORITY

ID	Requirement	Priority
F-TW-001	Translation workflows SHALL use ONLY models verified by LLMsVerifier.	Critical
F-TW-002	Model selection UI/API SHALL present only verified models with scoring information.	Critical
F-TW-003	Translation engine selection SHALL be restricted to providers/models passing LLMsVerifier gate.	Critical
F-TW-004	Batch translation SHALL validate all selected models against LLMsVerifier before execution.	High
F-TW-005	Distributed translation SHALL validate all worker-selected models against LLMsVerifier.	High
F-TW-006	SSH worker translations SHALL use only LLMsVerifier-approved models.	High
F-TW-007	Real-time translation via WebSocket SHALL use only verified models.	High
F-TW-008	Translation progress tracking SHALL include model verification status in metadata.	Medium

1.7 Data Flow and Synchronization - HIGH PRIORITY

ID	Requirement	Priority
F-DF-001	HelixTranslate SHALL synchronize model data with LLMsVerifier at configurable intervals.	High
F-DF-002		High

ID	Requirement	Priority
	Synchronization SHALL happen at: (a) application startup, (b) configurable periodic refresh, (c) on-demand via API/ admin trigger.	
F-DF-003	Incremental updates SHALL be supported to minimize data transfer.	Medium
F-DF-004	Full re-verification SHALL be triggerable on-demand.	Medium
F-DF-005	Synchronization events SHALL be logged and observable via WebSocket events.	Medium
F-DF-006	Model metadata changes SHALL trigger event notifications to subscribed HelixTranslate components.	Medium

1.8 Governance and Constitution Compliance - CRITICAL PRIORITY

ID	Requirement	Priority
F-GOV-001	Integration rules SHALL be added to HelixTranslate's `CONSTITUTION.md` mandating LLMsVerifier as SSOT.	Critical
F-GOV-002	Integration rules SHALL be added to HelixTranslate's `CLAUDE.md` with exact implementation guidance.	Critical
F-GOV-003	Integration rules SHALL be added to HelixTranslate's `AGENTS.MD` with operational procedures.	Critical
F-GOV-004	ALL submodules' `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.MD` files SHALL be updated with the same rules.	Critical
F-GOV-005	The anti-bluff testing mandate (CONST-035) SHALL be applied to all integration tests.	Critical
F-GOV-006	The host-power-management guard (CONST-033) SHALL be applied to all new scripts and challenges.	Critical
F-GOV-007	Reproduction-before-fix (CONST-032) SHALL be enforced for all integration bugs.	Critical
F-GOV-008	No mocks/fakes outside unit tests SHALL be the rule for all integration verification.	Critical

2. NON-FUNCTIONAL REQUIREMENTS

2.1 Performance Requirements

ID	Requirement	Priority
N-PERF-001	Model list retrieval from LLMsVerifier SHALL complete within 5 seconds for 25+ providers.	High
N-PERF-002	Model verification gate check SHALL add no more than 100ms latency to translation requests when using cached data.	High

ID	Requirement	Priority
N-PERF-003	Full model synchronization SHALL complete within 60 seconds.	High
N-PERF-004	HelixTranslate SHALL handle 1000+ verified models in memory efficiently.	High
N-PERF-005	Verification scoring computation SHALL be performed asynchronously and SHALL NOT block translation requests.	Critical
N-PERF-006	API calls to LLMsVerifier SHALL use connection pooling and keep-alive.	High
N-PERF-007	Go 1.25.2's performance characteristics SHALL be maintained; no regressions in throughput.	Critical
N-PERF-008	Memory usage for model cache SHALL be bounded with LRU eviction policy.	High
N-PERF-009	Translation throughput SHALL NOT degrade by more than 5% due to LLMsVerifier integration.	Critical
N-PERF-010	WebSocket event delivery SHALL maintain sub-100ms latency for model update notifications.	Medium

2.2 Security Requirements

ID	Requirement	Priority
N-SEC-001	API keys for LLMsVerifier SHALL be stored in HelixTranslate config file or .env file with 600 permissions (owner read/write only).	Critical
N-SEC-002	API keys SHALL NEVER be hardcoded, committed to git, or logged.	Critical
N-SEC-003	LLMsVerifier API communications SHALL use HTTPS/TLS in production.	Critical
N-SEC-004	Authentication tokens SHALL have configurable TTL and support refresh.	High
N-SEC-005	Model verification results SHALL be cryptographically signed to prevent tampering.	High
N-SEC-006	All configuration exports SHALL follow LLMsVerifier's security export pattern with warnings and gitignore protection.	Critical
N-SEC-007	Input validation SHALL be performed on all LLMsVerifier API responses.	High
N-SEC-008	SQL injection prevention SHALL be maintained for any database operations with model data.	High
N-SEC-009	Rate limiting SHALL prevent abuse of LLMsVerifier integration endpoints.	High
N-SEC-010	Security audit logging SHALL track all model verification decisions and provider changes.	High

2.3 Reliability and Availability Requirements

ID	Requirement	Priority
N-REL-001	LLMsVerifier integration SHALL have 99.9% uptime target for model retrieval operations.	High
N-REL-002	Graceful degradation SHALL occur when LLMsVerifier is unreachable - use cached verified models.	Critical
N-REL-003	Circuit breakers SHALL prevent cascading failures from LLMsVerifier unavailability.	Critical
N-REL-004	Health check endpoints SHALL expose LLMsVerifier connectivity status.	High
N-REL-005	Automatic recovery SHALL be attempted when LLMsVerifier becomes available again.	High
N-REL-006	No single point of failure SHALL be introduced by the LLMsVerifier integration.	Critical
N-REL-007	Retry policies SHALL be configurable: max retries, backoff strategy, timeout per attempt.	High

2.4 Scalability Requirements

ID	Requirement	Priority
N-SCL-001	Integration SHALL support horizontal scaling of HelixTranslate instances sharing LLMsVerifier data.	High
N-SCL-002	Distributed caching (Redis/cache layer) SHALL be supported for model data synchronization across instances.	Medium
N-SCL-003	Provider count SHALL be extensible beyond initial 25+ without code changes.	High
N-SCL-004	Model count per provider SHALL be unbounded in design.	High

2.5 Maintainability Requirements

ID	Requirement	Priority
N-MAINT-001	All integration code SHALL follow Go conventions, pass `gofmt`, and pass `golangci-lint`.	Critical
N-MAINT-002	Code SHALL follow existing HelixTranslate patterns (event-driven architecture, ~936 file structure).	Critical
N-MAINT-003	Dependency on LLMsVerifier SHALL use Go semantic versioning.	High
N-MAINT-004	Integration SHALL be versioned and support rolling updates.	High
N-MAINT-005	Feature flags SHALL allow gradual rollout of LLMsVerifier integration.	Medium
		High

ID	Requirement	Priority
N-MAINT-006	All code SHALL include comprehensive godoc comments.	
N-MAINT-007	Error handling SHALL use wrapped errors with context (per Go best practices and LLMsVerifier patterns).	High

3. INTEGRATION REQUIREMENTS

3.1 Code-Level Integration Points

ID	Requirement	Priority
I-CODE-001	**File: `go.mod`** - Add `digital.vasic.llmsverifier` as explicit dependency with version pinning.	Critical
I-CODE-002	**File: `internal/config/config.go`** - Add `LLMsVerifier` configuration struct with fields: `BaseURL`, `APIKey`, `Timeout`, `CacheTTL`, `StrictMode`, `VerificationEnabled`.	Critical
I-CODE-003	**File: `pkg/models/registry.go`** - Refactor `Registry` struct to use `EnhancedModelProviderService` from LLMsVerifier for model lookups. Remove hardcoded model definitions.	Critical
I-CODE-004	**File: `pkg/models/downloader.go`** - Add LLMsVerifier verification check before model download. Only download `CanSeeCode == true` models.	Critical
I-CODE-005	**File: `pkg/translator/*.go`** - Integrate LLMsVerifier client for model selection; reject unverified models.	Critical
I-CODE-006	**File: `pkg/api/*.go`** - Add `/api/v1/verified-models` endpoint returning LLMsVerifier-filtered models.	High
I-CODE-007	**File: `pkg/api/*.go`** - Add `/api/v1/verification-status` endpoint exposing LLMsVerifier sync status.	High
I-CODE-008	**File: `pkg/events/*.go`** - Emit events on model verification changes, provider additions/removals.	High
I-CODE-009	**File: `cmd/*.go`** - Add CLI commands for manual LLMsVerifier synchronization and verification status.	Medium
I-CODE-010	**File: `pkg/websocket/*.go`** - Broadcast model verification updates to connected clients.	Medium
I-CODE-011	**File: `internal/cache/*.go`** - Implement model data caching with TTL from LLMsVerifier.	High
I-CODE-012	**File: `pkg/models/errors.go`** - Add LLMsVerifier-specific error types: `ErrModelNotVerified`, `ErrVerificationFailed`, `ErrLLMsVerifierUnreachable`.	High

3.2 Data Model Integration

ID	Requirement	Priority
I-DM-001	HelixTranslate SHALL consume `VerifiedConfig` struct from LLMsVerifier's `verified_config_generator.go`.	Critical
I-DM-002	HelixTranslate's model representation SHALL be mappable from `VerifiedModelConfig` struct.	Critical
I-DM-003	Provider representation SHALL be mappable from `VerifiedProviderConfig` struct.	Critical
I-DM-004	Verification score SHALL be stored and exposed in HelixTranslate's model metadata.	High
I-DM-005	Feature flags (streaming, tool_calling, embeddings, vision, mcp, lsp, acp) SHALL be propagated from LLMsVerifier to HelixTranslate model objects.	Critical

3.3 Build and Dependency Integration

ID	Requirement	Priority
I-BUILD-001	`make build` SHALL include LLMsVerifier module compilation.	Critical
I-BUILD-002	`go mod tidy` SHALL resolve all LLMsVerifier dependencies cleanly.	Critical
I-BUILD-003	Docker build (`Dockerfile`) SHALL include LLMsVerifier module and its dependencies.	High
I-BUILD-004	Build verification SHALL confirm all LLMsVerifier providers are linked.	High

3.4 Runtime Integration Flow

ID	Requirement	Priority
I-RT-001	**Startup Flow**: (1) Load HelixTranslate config, (2) Initialize LLMsVerifier client with API key from config/.env, (3) Fetch verified models from LLMsVerifier, (4) Populate HelixTranslate registry with verified models only, (5) Begin periodic sync.	Critical
I-RT-002	**Translation Request Flow**: (1) User selects translation parameters, (2) HelixTranslate queries LLMsVerifier-enabled registry, (3) ONLY verified models presented, (4) User selects model, (5) Translation executes with verified model.	Critical
I-RT-003	**Model Update Flow**: (1) LLMsVerifier detects model changes, (2) Push/pull notification to HelixTranslate, (3) HelixTranslate refreshes model cache, (4) WebSocket broadcast to connected clients, (5) UI updates with new verified models.	High
I-RT-004	**Provider Addition Flow**: (1) New provider configured in LLMsVerifier, (2) Verification completes, (3) Provider	High

ID	Requirement	Priority
	appears in HelixTranslate after sync, (4) Models from new provider available for translation.	

3.5 HelixAgent Reference Integration Pattern

ID	Requirement	Priority
I-REF-001	HelixAgent's integration pattern (51+ providers, 35 MCPs) SHALL be studied and replicated for HelixTranslate.	Critical
I-REF-002	HelixAgent's submodule pattern (Agentic, Auth, Benchmark, Cache, Claritas, Concurrency, Containers, Database, Embeddings, EventBus, Formatters, HelixLLM, HelixMemory, HelixQA, etc.) SHALL inform HelixTranslate's architecture.	High
I-REF-003	API key management pattern from HelixAgent SHALL be replicated: config file or .env, 600 permissions, gitignore protection.	Critical
I-REF-004	HelixAgent's verification and challenge patterns SHALL be applied to HelixTranslate's LLMsVerifier integration.	Critical

4. TESTING REQUIREMENTS

4.1 Test Coverage Requirements - CRITICAL

ID	Requirement	Priority
T-COV-001	**100% test coverage** SHALL be achieved for ALL LLMsVerifier integration code.	Critical
T-COV-002	Unit tests SHALL cover: LLMsVerifier client initialization, config loading, model filtering, error handling, caching logic.	Critical
T-COV-003	Integration tests SHALL cover: full LLMsVerifier API communication, model retrieval, verification gate, config generation.	Critical
T-COV-004	E2E tests SHALL cover: complete translation workflow with LLMsVerifier-verified models only.	Critical
T-COV-005	ALL supported test types SHALL be implemented: unit, integration, E2E, automation, security/penetration, benchmark, chaos, stress, property-based, contract.	Critical
T-COV-006	**Anti-bluff testing (CONST-035)**: Tests MUST confirm codebase really works as expected, not just that test assertions pass.	Critical
T-COV-007	History of tests passing but features not working SHALL NOT repeat. Every test MUST verify actual behavior.	Critical
		Critical

ID	Requirement	Priority
T-COV-008	Test and Challenge execution MUST guarantee quality, completion, and full usability.	
T-COV-009	Mocks/stubs/fakes MAY ONLY be used in unit tests (`_test.go` files under `go test -short`). ALL other test types MUST use real LLMsVerifier instance.	Critical
T-COV-010	Fallback chains MUST be tested with actual failures (per Constitution).	Critical

4.2 Challenge Requirements - CRITICAL

ID	Requirement	Priority
T-CHAL-001	Challenge scripts SHALL be created in `challenges/scripts/` directory for ALL integration components.	Critical
T-CHAL-002	Challenge: `llmsverifier_connectivity_challenge.sh` - Verify HelixTranslate can connect to LLMsVerifier API.	Critical
T-CHAL-003	Challenge: `model_verification_gate_challenge.sh` - Confirm unverified models are rejected.	Critical
T-CHAL-004	Challenge: `verified_model_translation_challenge.sh` - Confirm translation works with verified models.	Critical
T-CHAL-005	Challenge: `provider_sync_challenge.sh` - Verify all 25+ providers synchronize correctly.	Critical
T-CHAL-006	Challenge: `api_key_provisioning_challenge.sh` - Verify API keys loaded from config/.env.	Critical
T-CHAL-007	Challenge: `mcp_integration_challenge.sh` - Verify MCP functionality with verified models.	Critical
T-CHAL-008	Challenge: `lsp_integration_challenge.sh` - Verify LSP functionality with verified models.	Critical
T-CHAL-009	Challenge: `acp_integration_challenge.sh` - Verify ACP functionality with verified models.	Critical
T-CHAL-010	Challenge: `embeddings_integration_challenge.sh` - Verify embeddings with verified models.	Critical
T-CHAL-011	Challenge: `rag_integration_challenge.sh` - Verify RAG functionality with verified models.	Critical
T-CHAL-012	Challenge: `skills_integration_challenge.sh` - Verify skills/plugins with verified models.	Critical
T-CHAL-013	Challenge: `cache_invalidation_challenge.sh` - Verify cache refresh triggers.	Critical
T-CHAL-014	Challenge: `degraded_mode_challenge.sh` - Verify graceful degradation when LLMsVerifier is down.	Critical
T-CHAL-015	Challenge: `host_power_management_challenge.sh` - Verify CONST-033 compliance.	Critical
		Critical

ID	Requirement	Priority
T-CHAL-016	Challenge: `no_suspend_calls_challenge.sh` - Verify no forbidden power management calls.	
T-CHAL-017	ALL challenges SHALL use real HTTP requests and live services (per Constitution).	Critical
T-CHAL-018	`./challenges/scripts/run_all_challenges.sh` SHALL execute ALL challenges and report pass/fail.	Critical

4.3 Anti-Bluff Testing Requirements - CRITICAL

ID	Requirement	Priority
T-AB-001	Tests MUST verify at the protocol layer: TCP open is the floor, NOT the ceiling.	Critical
T-AB-002	Postgres verification: execute `SELECT 1` and confirm result.	Critical
T-AB-003	Redis verification: execute `PING` and confirm `PONG` response.	Critical
T-AB-004	LLMsVerifier API: real HTTP request with real response and non-empty body.	Critical
T-AB-005	Model verification: confirm actual model verification status, not just API reachability.	Critical
T-AB-006	Container `Up` status is NOT sufficient - application must be actually healthy internally.	Critical
T-AB-007	Grep-only validation is NEVER sufficient.	Critical
T-AB-008	Every fix MUST be verified from all angles: runtime testing, compile verification, code structure checks, dependency checks, backward compatibility.	Critical
T-AB-009	Reproduction-before-fix (CONST-032): Write Challenge first, confirm fail, then fix, confirm pass.	Critical
T-AB-010	Evidence MUST be pasted from actual command runs in the same session as changes.	Critical

4.4 Testing Infrastructure Requirements

ID	Requirement	Priority
T-INF-001	Resource limits: ALL test/challenge execution limited to 30-40% host resources (`GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1`).	Critical
T-INF-002	Test database: Use real SQLite with SQL Cipher (per LLMsVerifier pattern).	High
T-INF-003	Test LLMsVerifier instance SHALL be available for integration tests.	Critical
T-INF-004	Parallel test execution SHALL be configurable and resource-bounded.	High

ID	Requirement	Priority
T-INF-005	Test reports SHALL be generated in multiple formats: JSON, HTML, JUnit XML.	Medium
T-INF-006	Coverage reports SHALL be generated with `coverage.out`, `coverage.html`.	High
T-INF-007	Benchmark tests SHALL measure: model retrieval latency, verification gate latency, sync throughput.	High

5. DOCUMENTATION REQUIREMENTS

5.1 Primary Documentation - CRITICAL

ID	Requirement	Priority
D-DOC-001	**Integration Architecture Document**: Complete architecture diagram showing HelixTranslate <-> LLMsVerifier data flow, component interactions, and integration points.	Critical
D-DOC-002	**API Integration Guide**: Document all LLMsVerifier API endpoints used, request/response formats, error codes, retry logic.	Critical
D-DOC-003	**Configuration Reference**: EVERY single configuration option in main HelixTranslate config file (or .env) MUST be documented with: name, type, default value, description, example, validation rules.	Critical
D-DOC-004	**Migration Guide**: Step-by-step guide for migrating from hardcoded models to LLMsVerifier SSOT.	Critical
D-DOC-005	**Developer Guide**: How to add new providers, modify verification criteria, extend scoring.	Critical
D-DOC-006	**Troubleshooting Guide**: Common issues, diagnostics, resolution steps for LLMsVerifier integration.	High
D-DOC-007	**Security Guide**: API key management, authentication, encryption, secure configuration practices.	Critical

5.2 Constitution and Governance Documentation - CRITICAL

ID	Requirement	Priority
D-GOV-001	`CONSTITUTION.md` SHALL contain explicit clause mandating LLMsVerifier as SSOT for all models.	Critical
D-GOV-002	`CLAUDE.md` SHALL contain exact implementation guidance for LLMsVerifier integration with codebase references.	Critical
D-GOV-003	`AGENTS.MD` SHALL contain operational procedures for LLMsVerifier integration maintenance.	Critical
		Critical

ID	Requirement	Priority
D-GOV-004	ALL submodule `CONSTITUTION.md` files SHALL be updated with the same LLMsVerifier SSOT mandate.	
D-GOV-005	ALL submodule `CLAUDE.md` files SHALL be updated with LLMsVerifier integration guidance.	Critical
D-GOV-006	ALL submodule `AGENTS.MD` files SHALL be updated with operational procedures.	Critical
D-GOV-007	Documentation SHALL reference exact file paths, function names, and line numbers where applicable.	High

5.3 User Documentation - HIGH PRIORITY

ID	Requirement	Priority
D-USER-001	**End-User Guide**: How to select verified models for translation, interpret verification scores, understand feature indicators.	High
D-USER-002	**Admin Guide**: How to configure LLMsVerifier integration, manage API keys, trigger manual sync, view verification status.	High
D-USER-003	**Website Documentation** (`Website/` directory): Update public-facing docs to mention LLMsVerifier integration.	Medium
D-USER-004	Video course content SHALL be updated if it covers model selection (reference: LLMsVerifier's `ACP_VIDEO_COURSE_CONTENT.md`).	Low

5.4 Testing Documentation - CRITICAL

ID	Requirement	Priority
D-TEST-001	**Test Strategy Document**: Complete testing strategy for LLMsVerifier integration covering ALL test types.	Critical
D-TEST-002	**Challenge Documentation**: Each challenge SHALL have documentation explaining what it tests, how to run it, expected output.	Critical
D-TEST-003	**Test Results Documentation**: Format for reporting test results with pass/fail, coverage percentage, anti-bluff verification.	High
D-TEST-004	`TESTING_FRAMEWORK_SPECIFICATION.md` SHALL be updated with LLMsVerifier integration test specifications.	Critical

5.5 API Documentation - HIGH PRIORITY

ID	Requirement	Priority
D-API-001	New REST endpoints (`/api/v1/verified-models`, `/api/v1/verification-status`) SHALL be documented with OpenAPI/Swagger specs.	High

ID	Requirement	Priority
D-API-002	WebSocket events for model updates SHALL be documented with message schemas.	High
D-API-003	gRPC protocol definitions SHALL be updated if applicable (reference `pkg/grpc/`).	Medium

6. UX REQUIREMENTS

6.1 Enterprise-Grade Translation Tool UX - CRITICAL

ID	Requirement	Priority
UX-001	Model selection interface SHALL display ONLY verified models with clear verification badges.	Critical
UX-002	Each model SHALL display: verification score (0-100), last verified timestamp, supported features icons, cost per 1M tokens.	Critical
UX-003	Models SHALL be sortable by: verification score, cost, response time, feature count.	High
UX-004	Models SHALL be filterable by: provider, features (streaming, tool calling, vision, embeddings, MCP, LSP, ACP), price range, verification status.	High
UX-005	Unverified or failed models SHALL be completely hidden from end users (not grayed out - invisible).	Critical
UX-006	If no verified models are available for a provider, that provider SHALL be marked as "No verified models" or hidden.	High
UX-007	Verification score SHALL be color-coded: green (80-100), yellow (50-79), orange (20-49), red (0-19).	Medium
UX-008	Feature support SHALL be displayed as icon grid with hover tooltips explaining each feature.	Medium
UX-009	Cost comparison SHALL be available across verified models for the same translation task.	Medium
UX-010	Translation quality estimation SHALL be shown based on model's verification score and features.	Medium

6.2 Real-Time Feedback - HIGH PRIORITY

ID	Requirement	Priority
UX-011	Real-time model sync status SHALL be visible in UI (last sync time, next sync time, sync health).	High
UX-012	New verified models SHALL appear in UI without page refresh (WebSocket push).	High
UX-013	Model verification failures SHALL trigger user-visible notifications with explanation.	Medium

ID	Requirement	Priority
UX-014	Degraded mode (LLMsVerifier unreachable) SHALL show clear warning banner with cache age indicator.	High

6.3 Professional Translation Workflow - HIGH PRIORITY

ID	Requirement	Priority
UX-015	Translation job setup SHALL include model recommendation based on: language pair, document type, quality requirements, budget.	High
UX-016	Batch translation SHALL show per-model verification status in job configuration.	Medium
UX-017	Distributed translation SHALL show verification status of models on each worker node.	Medium
UX-018	Translation progress SHALL include model name and verification score in progress metadata.	Low
UX-019	Post-translation report SHALL include which verified model was used and its performance metrics.	Medium

6.4 Accessibility and Internationalization - MEDIUM PRIORITY

ID	Requirement	Priority
UX-020	All verification status indicators SHALL be accessible (ARIA labels, screen reader compatible).	Medium
UX-021	Verification score display SHALL support screen readers with verbal description.	Medium
UX-022	UI text SHALL be internationalized for supported languages.	Low

7. CONFIGURATION REQUIREMENTS

7.1 LLMsVerifier Configuration Section - CRITICAL

The following configuration options MUST be available in HelixTranslate's main config file (config.json or .env). EVERY option MUST be documented.

ID	Config Option	Type	Default	
CFG-001	`llmsverifier.enabled`	boolean	`true`	
CFG-002	`llmsverifier.base_url`	string	`"http://localhost:8080"`	

ID	Config Option	Type	Default
CFG-003	`llmsverifier.api_key`	string	`""`
CFG-004	`llmsverifier.timeout_seconds`	integer	`30`
CFG-005	`llmsverifier.cache_ttl_minutes`	integer	`60`
CFG-006	`llmsverifier.strict_mode`	boolean	`true`
CFG-007	`llmsverifier.verification_enabled`	boolean	`true`
CFG-008	`llmsverifier.sync_interval_minutes`	integer	`30`
CFG-009	`llmsverifier.retry_max_attempts`	integer	`3`
CFG-010	`llmsverifier.retry_backoff_seconds`	integer	`5`
CFG-011	`llmsverifier.circuit_breaker_threshold`	integer	`5`
CFG-012	`llmsverifier.circuit_breaker_timeout_seconds`	integer	`60`
CFG-013	`llmsverifier.min_verification_score`	float	`0.0`
CFG-014	`llmsverifier.helixagent_api_key`	string	`""`

ID	Config Option	Type	Default	
CFG-015	`llmsverifier.helixagent_base_url`	string	`""`	

7.2 Provider Configuration - CRITICAL

ID	Config Option	Type	Default	Description	Priority
CFG-016	`providers.*.enabled`	boolean	`true`	Enable/disable specific provider	High
CFG-017	`providers.*.api_key`	string	`""`	Provider-specific API key (from .env)	Critical
CFG-018	`providers.*.api_key_env`	string	`""`	Environment variable name for provider API key	Critical
CFG-019	`providers.*.base_url`	string	`""`	Provider-specific API base URL override	High
CFG-020	`providers.*.rate_limit_rpm`	integer	`0`	Rate limit (requests per minute), 0 = unlimited	High
CFG-021	`providers.*.timeout_seconds`	integer	`30`	Provider-specific timeout	Medium
CFG-022	`providers.*.model_filter`	string[]	`[]`	Whitelist of model IDs to include (empty = all verified)	Medium
CFG-023	`providers.*.require_verification`	boolean	`true`	If false, skip verification for this provider (not recommended)	Critical

7.3 Feature Flags - HIGH PRIORITY

ID	Config Option	Type	Default	Description	Priority
CFG-024	`features.mcp_enabled`	boolean	`true`	Enable MCP integration	High

ID	Config Option	Type	Default	Description	Priority
CFG-025	`features.lsp_enabled`	boolean	`true`	Enable LSP integration	High
CFG-026	`features.acp_enabled`	boolean	`true`	Enable ACP integration	High
CFG-027	`features.embeddings_enabled`	boolean	`true`	Enable embeddings integration	High
CFG-028	`features.rag_enabled`	boolean	`true`	Enable RAG integration	High
CFG-029	`features.skills_enabled`	boolean	`true`	Enable skills integration	High
CFG-030	`features.plugins_enabled`	boolean	`true`	Enable plugins integration	High
CFG-031	`features.streaming_enabled`	boolean	`true`	Require streaming support for models	High
CFG-032	`features.tool_calling_enabled`	boolean	`true`	Require tool calling support for models	High
CFG-033	`features.vision_enabled`	boolean	`false`	Require vision support for models	Medium

7.4 Environment Variable Mapping - CRITICAL

ID	Environment Variable	Maps To	Priority
CFG-ENV-001	`HELIX_LLV_ENABLED`	`llmsverifier.enabled`	Critical
CFG-ENV-002	`HELIX_LLV_BASE_URL`	`llmsverifier.base_url`	Critical
CFG-ENV-003	`HELIX_LLV_API_KEY`	`llmsverifier.api_key`	Critical
CFG-ENV-004	`HELIX_LLV_TIMEOUT`	`llmsverifier.timeout_seconds`	High
CFG-ENV-005	`HELIX_LLV_CACHE_TTL`	`llmsverifier.cache_ttl_minutes`	High
CFG-ENV-006	`HELIX_LLV_STRICT_MODE`	`llmsverifier.strict_mode`	Critical
CFG-ENV-007	`HELIX_LLV_SYNC_INTERVAL`	`llmsverifier.sync_interval_minutes`	High

ID	Environment Variable	Maps To	Priority
CFG-ENV-008	HELIX_AGENT_API_KEY	llmsverifier.helixagent_api_key	Critical
CFG-ENV-009	HELIX_AGENT_BASE_URL	llmsverifier.helixagent_base_url	Critical
CFG-ENV-010	ANTHROPIC_API_KEY	providers.anthropic.api_key	Critical
CFG-ENV-011	OPENAI_API_KEY	providers.openai.api_key	Critical
CFG-ENV-012	GROQ_API_KEY	providers.groq.api_key	Critical
CFG-ENV-013	DEEPSEEK_API_KEY	providers.deepseek.api_key	Critical
CFG-ENV-014	MISTRAL_API_KEY	providers.mistral.api_key	Critical
CFG-ENV-015	COHERE_API_KEY	providers.cohere.api_key	Critical
CFG-ENV-016	TOGETHERAI_API_KEY	providers.togetherai.api_key	Critical
CFG-ENV-017	XAI_API_KEY	providers.xai.api_key	Critical
CFG-ENV-018	HYPERBOLIC_API_KEY	providers.hyperbolic.api_key	Critical
CFG-ENV-019	SAMBANOVA_API_KEY	providers.sambanova.api_key	Critical
CFG-ENV-020	CEREBRAS_API_KEY	providers.cerebras.api_key	Critical
CFG-ENV-021	CLOUDFLARE_API_KEY	providers.cloudflare.api_key	Critical
CFG-ENV-022	KIMI_API_KEY	providers.kimi.api_key	Critical
CFG-ENV-023	NOVITA_API_KEY	providers.novita.api_key	Critical
CFG-ENV-024	QWEN_API_KEY	providers.qwen.api_key	Critical
CFG-ENV-025	REPLICATE_API_KEY	providers.replicate.api_key	Critical
CFG-ENV-026	SILICONFLOW_API_KEY	providers.siliconflow.api_key	Critical
CFG-ENV-027	UPSTAGE_API_KEY	providers.upstage.api_key	Critical

8. CONSTRAINTS AND ASSUMPTIONS

8.1 Technical Constraints

ID	Constraint	Impact
C-001	**Go 1.25.2**: HelixTranslate uses Go 1.25.2. LLMsVerifier integration MUST be compatible.	Build compatibility
C-002	**No CI/CD Pipelines**: Per Constitution, no GitHub Actions, GitLab CI, Jenkins, etc. All builds manual or Makefile.	Build process
C-003	**SSH URLs Only**: No HTTPS for Git. SSH URLs only for all submodule operations.	Repository access
C-004	**No Manual Container Commands**: Container orchestration owned by project binary/orchestrator.	Deployment
C-005	**Resource Limits**: ALL tests/challenges limited to 30-40% host resources.	Test execution
C-006	**No Mocks in Production**: Mocks/stubs/fakes ONLY in unit tests. Real services for all other tests.	Testing approach
C-007	**SQLite with SQL Cipher**: Database encryption required for any local model data storage.	Data storage
C-008	**Event-Driven Architecture**: HelixTranslate is event-driven. Integration MUST emit/consume events.	Architecture
C-009	**HTTP/3 Support**: REST API has HTTP/3 support. LLMsVerifier client SHOULD support HTTP/2 at minimum.	Protocol
C-010	**WebSocket Events**: Real-time events via WebSocket. Model updates MUST be WebSocket-broadcast.	Real-time updates

8.2 Business Constraints

ID	Constraint	Impact
C-011	**Single Source of Truth**: LLMsVerifier is the ONLY source. No hardcoded fallbacks allowed.	Design approach
C-012	**Enterprise-Grade UX**: Must be cutting-edge professional translation tool.	UX investment
C-013	**Heavy Translation Use Case**: Must support all providers and models for intensive translation workloads.	Performance
C-014	**No Fabricated Results**: Anti-bluff mandate. Real verification only.	Quality assurance
C-015	**Host Power Management**: CONST-033 prohibits any host power state transitions.	System stability

8.3 Assumptions

ID	Assumption	Risk if Invalid
A-001	LLMsVerifier instance is accessible at configured BaseURL.	Degraded mode activation
A-002	LLMsVerifier has completed at least one full verification cycle.	No models available
A-003	API keys for providers are valid and have sufficient quota.	Translation failures
A-004	HelixAgent (reference) integration pattern is stable and correct.	Design divergence
A-005	Network connectivity between HelixTranslate and LLMsVerifier is reliable.	Cache dependency
A-006	Go module `digital.vasic.llmsverifier` is publishable/importable.	Dependency resolution

9. SUCCESS CRITERIA

9.1 Functional Success Criteria - MEASURABLE

ID	Criterion	Measurement
S-F-001	ALL translation requests use ONLY LLMsVerifier-verified models.	100% of `/api/v1/translate` calls use verified models. Log audit confirms.
S-F-002	Zero unverified models are presented to end users.	UI/API response inspection shows only `VerificationStatus == "verified"` models.
S-F-003	All 25+ LLMsVerifier providers are available in HelixTranslate.	Provider count in `/api/v1/providers` >= 25.
S-F-004	MCP, LSP, ACP, Embeddings, RAG, Skills, Plugins are all functional.	Each feature has passing challenge script.
S-F-005	API keys for LLMsVerifier and HelixAgent are configurable via config/.env.	Config loading tests pass with all 27+ environment variables.
S-F-006	Model sync occurs automatically at configured intervals.	WebSocket events confirm sync. Log timestamps match interval.

9.2 Integration Success Criteria - MEASURABLE

ID	Criterion	Measurement
S-I-001	LLMsVerifier Go module is importable and builds successfully.	`go build` completes with zero errors. `go mod tidy` resolves cleanly.
S-I-002	`pkg/models/registry.go` delegates to LLMsVerifier with no hardcoded models.	Code review confirms no hardcoded model lists in registry.
S-I-003	`internal/config/config.go` includes all 33+ LLMsVerifier configuration options.	Config test enumerates all options and validates defaults.
S-I-004	All integration code follows HelixTranslate event-driven patterns.	Code review confirms event emission/consumption for model changes.
S-I-005	Docker build includes LLMsVerifier dependencies.	`docker build` succeeds and image runs with LLMsVerifier integration.

9.3 Testing Success Criteria - MEASURABLE

ID	Criterion	Measurement
S-T-001	100% code coverage for LLMsVerifier integration code.	`coverage.out` shows 100% for integration package.
S-T-002	ALL 17+ challenge scripts pass.	`./challenges/scripts/run_all_challenges.sh` exits 0.
S-T-003	Anti-bluff tests confirm real functionality.	Challenge logs show actual HTTP requests with real responses.
S-T-004	No test passes while feature is broken.	Deliberate bug injection causes test failure.
S-T-005	Unit tests use mocks only; integration+ tests use real services.	Code review of `_test.go` files confirms pattern.
S-T-006	Resource limits respected during test execution.	`htop`/`ps` confirms < 40% resource usage during test runs.

9.4 Documentation Success Criteria - MEASURABLE

ID	Criterion	Measurement
S-D-001	ALL 33+ configuration options are documented.	Documentation checklist confirms every CFG-* item has corresponding docs.
S-D-002	`CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.MD` in HelixTranslate contain LLMsVerifier SSOT clause.	File search finds explicit LLMsVerifier SSOT mandate.

ID	Criterion	Measurement
S-D-003	ALL submodule governance docs updated.	Script verifies all submodule `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.MD` contain clause.
S-D-004	Integration Architecture Document exists with data flow diagrams.	Document review confirms architecture diagrams and flow descriptions.

9.5 UX Success Criteria - MEASURABLE

ID	Criterion	Measurement
S-UX-001	End users see only verified models with scores and feature icons.	Screenshot/UI inspection confirms no unverified models visible.
S-UX-002	Model selection includes sorting, filtering, and recommendation.	UI test confirms sort/filter/recommendation functionality.
S-UX-003	Real-time model updates appear without page refresh.	WebSocket message triggers UI update within 2 seconds.
S-UX-004	Degraded mode shows clear warning when LLMsVerifier unreachable.	Network partition test confirms warning banner display.
S-UX-005	Enterprise-grade professional appearance throughout.	Design review against enterprise UX standards.

REQUIREMENTS TRACEABILITY MATRIX

User Requirement	Functional	Non-Functional	Integration	Testing	Docum
SSOT for all models	F-SSOT-001..007	N-REL-001..007	I-CODE-001..012	T-COV-001..010	D-DOC-0
Only validated/verified/scored models	F-GATE-001..009	N-SEC-001..010	I-DM-001..005	T-CHAL-001..018	D-TEST-0
Step-by-step codebase guide	F-GOV-001..008	N-MAINT-001..007	I-CODE-001..012, I-REF-001..004	T-COV-001..010, T-AB-001..010	D-GOV-0
In-depth repo analysis	-	-	I-REF-001..004	-	D-DOC-0
Rock-solid bluff-proof plan	F-GATE-001..009	N-REL-001..007, N-SEC-001..010	I-RT-001..004	T-AB-001..010, T-COV-006..010	D-TEST-0
Phased fine-grained tasks	All functional	All non-functional	All integration	All testing	All docum
Comprehensive, nothing skipped	All requirements	All requirements	All requirements	All requirements	All require

User Requirement	Functional	Non-Functional	Integration	Testing	Docun
Enterprise-grade UX	F-TW-001..008	N-PERF-001..010	I-RT-001..004	T-CHAL-002..004	D-USER-
All providers incorporated	F-PROV-001..007	N-SCL-001..004	I-CODE-003..005	T-CHAL-005	D-DOC-0
All MCPs/ LSPs/ACPs/ Embeddings/ RAGs/Skills/ Plugins	F-EXT-001..009	N-PERF-001..010	I-CODE-006..012	T-CHAL-007..012	D-DOC-0
API keys via config/.env	F-API-005	N-SEC-001..003	I-CODE-002	T-CHAL-006	D-DOC
ALL documentation prepared	F-GOV-001..008	N-MAINT-001..007	I-CODE-001..012	T-COV-001..010	D-DOC API-00
Every config option documented	-	-	I-CODE-002	-	D-DOC
Testing strategy 100% coverage	F-GATE-001..009	N-REL-001..007	I-RT-001..004	T-COV-001..010, T-CHAL-001..018, T-AB-001..010	D-TEST-0
Anti-bluff testing	-	-	-	T-AB-001..010, T-COV-006..010	D-TEST-0
Constitution/ CLAUDE.MD/ AGENTS.MD	F-GOV-001..008	N-MAINT-001..007	-	-	D-GOV-0
Submodules' docs updated	F-GOV-004..008	N-MAINT-001..007	-	-	D-GOV-0

PHASED IMPLEMENTATION STRUCTURE

Phase 1: Foundation and Analysis (Prerequisites)

- **1.1** Complete codebase analysis of ALL three repositories
- **1.2** Map all integration points with exact file paths and line references
- **1.3** Define data contracts between HelixTranslate and LLMsVerifier
- **1.4** Set up LLMsVerifier module dependency in HelixTranslate go.mod
- **1.5** Create feature branch for integration work

Phase 2: Core Integration Development

- **2.1** Implement LLMsVerifier client wrapper in HelixTranslate
- **2.2** Add configuration system (33+ config options)

- **2.3** Refactor pkg/models/registry.go to use LLMsVerifier SSOT
- **2.4** Implement verification gate in model selection flow
- **2.5** Implement caching with TTL and invalidation
- **2.6** Add error handling and circuit breakers

Phase 3: Provider and Feature Expansion

- **3.1** Integrate all 25+ LLMsVerifier providers
- **3.2** Integrate MCP support (35 MCPs from HelixAgent reference)
- **3.3** Integrate LSP support
- **3.4** Integrate ACP support
- **3.5** Integrate Embeddings support
- **3.6** Integrate RAG support
- **3.7** Integrate Skills and Plugins support

Phase 4: API and Real-Time Integration

- **4.1** Add REST endpoints for verified models and verification status
- **4.2** Add WebSocket events for model updates
- **4.3** Update gRPC protocols if applicable
- **4.4** Integrate with translation workflow (single, batch, distributed, SSH)

Phase 5: UX Implementation

- **5.1** Implement verified model display with scores and features
- **5.2** Implement sorting, filtering, recommendation
- **5.3** Implement real-time update UI
- **5.4** Implement degraded mode warning
- **5.5** Accessibility compliance

Phase 6: Testing and Quality Assurance

- **6.1** Unit tests (100% coverage, mocks only)
- **6.2** Integration tests (real LLMsVerifier instance)
- **6.3** E2E tests (full translation workflows)
- **6.4** Challenge scripts (17+ challenges)
- **6.5** Anti-bluff verification
- **6.6** Security/penetration tests
- **6.7** Benchmark and performance tests
- **6.8** Chaos and stress tests

Phase 7: Documentation and Governance

- **7.1** Update CONSTITUTION.md with LLMsVerifier SSOT mandate
- **7.2** Update CLAUDE.md with implementation guidance
- **7.3** Update AGENTS.MD with operational procedures
- **7.4** Update ALL submodule governance docs
- **7.5** Create integration architecture document
- **7.6** Create API documentation
- **7.7** Create configuration reference
- **7.8** Create user and admin guides

Phase 8: Validation and Deployment

- **8.1** Run full validation suite
- **8.2** Run all challenges
- **8.3** Verify 100% coverage
- **8.4** Verify anti-bluff compliance
- **8.5** Docker build verification
- **8.6** Deploy to staging
- **8.7** Production deployment with monitoring

END OF REQUIREMENTS DOCUMENT Total Requirements: 300+ individual requirements across 9 categories All explicit and implicit requirements from user query have been captured and categorized.